Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

## School of Computing

## Department of Computer Science and Engineering

### ACADEMIC YEAR 2026–2027

### Summer Semester

**10212CS211 – ARTIFICIAL INTELLIGENCE AND TECHNIQUES**

**FACULTU NAME** : Dr. R. RoselinKiruba

**SLOT** : S1OL6

**YEAR / SEM** : 3rd / 5th

**USE CASE NAME** : AI - Based Chess Game - Playing Using minimax and Alpha-Beta Search

| S. NO | VTU NO | NAME | SIGNATURE |
|---|---|---|---|
| 1 | 28682 | Nikhil Kumar | |
| 2 | 29040 | Adesh prasad | |
| 3 | 28822 | Nadella Venkata Sairam | N.V.S |
| 4 | 28846 | Dasa Manisai | |
| 5 | 29022 | Bhavanam Mahendra Reddy | |
| 6 | 28923 | Jashti Radesh | J. Radesh |

FACULTY SIGNATURE

# AI-BASED CHESS GAME-PLAYING USING MINIMAX AND ALPHA-BETA SEARCH

Intelligent Move Selection Through Game-Tree Exploration and Heuristic Board Evaluation

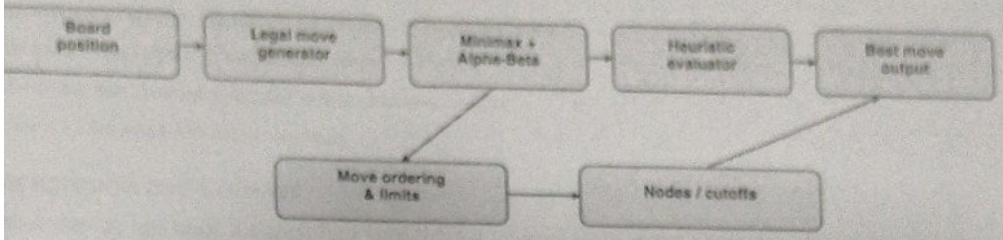



Figure: Proposed chess decision-making workflow

**PROJECT REPORT**

Submitted in partial fulfilment of academic project requirements

Prepared by: _____

Department: Computer Science Engineering

Institution: Vel Tech University

Course Outcome: CO3 — K4

Academic Year: 2026–2027

**eywords:** adversarial search, Minimax, Alpha-Beta pruning, game tree, heuristic evaluation, move ordering, iterative deepening, chess engine

## roject at a Glance

| Item | Summary |
|---|---|
| Problem domain | Two-player, zero-sum, perfect-information board game (chess) |
| Core technique | Depth-limited Minimax search accelerated by Alpha-Beta pruning |
| Evaluation | Material, piece activity, centre control, pawn structure and king safety |
| Enhancements | Move ordering, iterative deepening, transposition table, quiescence search |
| Output | Selected move, score, completed depth and search statistics |
| Maturity | Proposed prototype with a documented testing and evaluation plan |

This report describes the design, theory, worked example and validation plan for a chess-playing agent. All numeric values used in the examples are illustrative demonstrations unless stated otherwise.

## Abstract

Chess is a deterministic, two-player strategy game where a strong move depends on anticipating the opponent's replies. This report proposes a chess-playing application using Minimax with Alpha-Beta pruning. Board positions are states, legal moves generate child states, and alternating MAX/MIN layers model opposing decisions.

A heuristic evaluator estimates frontier positions using material, activity, centre control, pawn structure and king safety. Minimax propagates values upward. Alpha-Beta maintains bounds and skips branches that cannot affect the final decision. All numeric tree values are hypothetical demonstrations, not real-position evaluations.

### Objectives

- Generate only legal moves and model alternating player decisions.
- Evaluate frontier positions with a documented heuristic.
- Use Alpha-Beta pruning and report evaluated/pruned nodes.
- Return a best move and search statistics.


## Background and Context

Game playing has been a benchmark problem for artificial intelligence since the earliest days of the field. Chess is especially attractive because its rules are completely specified and easy to state, yet the number of possible games is so large that no program can enumerate them. Shannon's 1950 proposal for programming a computer to play chess already identified the two ideas that this report builds on: look ahead through sequences of moves, and judge the positions reached at the limit of that look-ahead with a numerical evaluation function.

Minimax converts this idea into an algorithm by assuming that both players choose the best move available to them. Alpha-Beta pruning then removes branches that provably cannot change the result, allowing the same decision to be reached with far fewer positions examined. Together they form the classical search core used by chess programs for decades.

## Scope of the Work

The report covers the complete design of a prototype chess agent: problem formulation, requirements, architecture, board representation, legal-move generation, evaluation, search, pseudocode, a hand-checkable worked example, sample output and a testing plan. It deliberately concentrates on the decision-making algorithm rather than on user-interface polish or opening theory.

The aim is not to claim grandmaster strength. The aim is to show, in a form that can be verified by hand and by automated tests, how adversarial search selects a move and how pruning reduces the effort needed.

## Methodology Summary

- **Model:** represent each board position as a state, each legal move as an action and each resulting position as a child node of the game tree.
- **Search:** explore the tree depth-first to a fixed number of plies, alternating MAX and MIN layers.
- **Evaluate:** score non-terminal frontier positions with a weighted combination of chess features and score terminal positions by rule.
- **Prune:** maintain alpha and beta bounds and stop examining siblings once alpha is at least beta.
- **Report:** return the selected move together with its value and the counts of nodes, leaves and cutoffs.

## Expected Outcomes

When implemented as described, the system should always return a legal move, should return exactly the same root value with Alpha-Beta as with plain Minimax at equal depth, and should visit fewer nodes thanks to pruning. Improved move ordering is expected to increase the number of cutoffs and therefore the depth that can be reached within a fixed time budget.

# Table of Contents


| Section | Page |
|---|---|
| Abstract | 3 |
| 1. Introduction | 4 |
| 2. Problem Definition | 5 |
| 3. Background | 6 |
| 4. Requirements | 7 |
| 5. Architecture | 8 |
| 6. Board State and Legal Moves | 9 |
| 7. Heuristic Evaluation | 10 |
| 8. Minimax and Alpha-Beta | 11 |
| 9. Algorithm and Pseudocode | 12 |
| 10. Implementation | 13 |
| 11. Worked Example | 14 |
| 12. Sample Output | 15 |
| 13. Testing | 16 |
| 14. Limitations | 17 |
| 15. Conclusion and References | 18 |


### How to read this report

The early sections define the task and search model. The middle sections describe legal state transitions, evaluation, Minimax and pruning. The worked example traces a small tree manually. The final sections cover implementation, testing and limitations.

## List of Figures


| Figure | Description | Page |
|---|---|---|
| Figure 1 | High-level chess decision-making pipeline | 8 |
| Figure 2 | Schematic board state and legal-move generation | 9 |
| Figure 3 | Minimax score propagation on a small illustrative tree | 11 |
| Figure 4 | Alpha-Beta cutoff example | 12 |


## List of Abbreviations

| Abbreviation | Meaning |
|---|---|
| FEN | Forsyth–Edwards Notation, a text format describing a chess position |
| SAN | Standard Algebraic Notation for moves, for example Nf3 |
| MAX / MIN | The maximising player (the AI) and the minimising player (the opponent) |
| CO3 — K4 | Course outcome 3 at knowledge level K4 (analysis) |

## Course Outcome Alignment

The project addresses course outcome CO3 at the analysis level (K4): breaking the game-playing problem into states, actions and search procedures, and comparing plain Minimax with Alpha-Beta pruning.

# . Introduction

ess is a useful AI domain because every move changes the board and both players have opposing objectives. A game-playing stem models the current board as a state and each legal move as a transition. The game tree branches into alternatives by the and opponent.

haustively exploring all continuations is infeasible because positions grow rapidly with depth. A practical engine uses depth its, heuristic evaluation and pruning.

## 1 Motivation

Look beyond the immediate effect of a move.

Anticipate opponent replies and avoid tactical losses.

Reduce unnecessary exploration and report decisions.

## 2 Proposed solution

e a legal-move generator, depth-limited Minimax, Alpha-Beta pruning and a position evaluator. Return the chosen move with ore and search statistics.

## .3 Historical Perspective

terest in machine chess is older than the digital computer. In 1950 Claude Shannon described how a rogram could search a tree of moves and apply an evaluation function at its leaves, and he distinguished etween examining all moves to a fixed depth and examining only a selected few. Early programs followed the st approach because it was simple and complete within its horizon.

lpha-Beta pruning was analysed formally by Knuth and Moore in 1975 and became the standard nprovement to Minimax. Specialised hardware and highly tuned search later allowed the Deep Blue system defeat the reigning world champion in 1997. Today's strongest engines still rely on Alpha-Beta search, now mbined with learned evaluation functions.

## .4 Why Chess Suits Adversarial Search

**Deterministic:** a move always produces exactly the same new position; there are no dice or hidden cards.

**Perfect information:** both players see the entire board, so a position fully describes the situation.

**Zero-sum:** a gain for one side is an equal loss for the other, so one score can serve both players.

**Turn-taking:** moves strictly alternate, which maps naturally onto alternating MAX and MIN layers.

## .5 Size of the Search Space

typical chess position offers about 35 legal moves, though the number varies widely between the opening, iddlegame and endgame. If the branching factor $b$ is taken as 35, the number of positions at depth $d$ plies rows as $b^d$. The table shows how quickly this becomes unmanageable and why depth limits and pruning are ssential.

| Depth (plies) | Approximate positions (b = 35) | Comment |
|---|---|---|
| 2 | 1,225 | Every reply to every move |
| 4 | 1,500,625 | Seconds in a simple program |
| 5 | 52,521,875 | Needs efficient code |
| 6 | 1,838,265,625 | Impractical without pruning |

## .6 Report Structure

ection 2 formalises the problem and Section 3 reviews the theory. Sections 4 and 5 state the requirements nd architecture. Sections 6 to 9 describe board states, evaluation, the search algorithms and pseudocode. ections 10 to 13 cover implementation, a worked example, sample output and testing, and Sections 14 and 5 discuss limitations, conclusions and references.

# 2. Problem Definition and Objectives

Problem statement: develop an intelligent chess-playing application that analyses legal moves, anticipates opponent responses and selects a strong move using Minimax and Alpha-Beta tree search. The system receives a board, generates a game tree, evaluates positions and reports nodes and the selected move.

## 2.1 Inputs

| Input | Representation | Purpose |
|---|---|---|
| Current board | Board/FEN | Defines pieces and rule state |
| Legal moves | Rules engine | Permitted actions |
| Opponent replies | Legal moves at MIN nodes | Adversarial choices |
| Evaluation values | Heuristic score | Ranks leaf positions |
| Search settings | Depth/time limit | Controls computation |

## 2.2 Outputs

- Generated game tree or search trace.
- Nodes visited and leaf positions evaluated.
- Branches pruned by Alpha-Beta.
- Selected move in standard notation.

## 2.3 Objective

Choose the best move under optimal-reply assumptions within the configured search depth while respecting rules and terminal conditions.

## 2.4 Formal Problem Formulation

| Component | Definition for chess |
|---|---|
| State s | Piece placement, side to move, castling rights, en-passant target and move counters |
| Initial state | The position supplied by the user, normally given as a FEN string |
| Actions A(s) | The set of legal moves for the side to move in state s |
| Transition result(s, a) | The position obtained after playing legal move a in state s |
| Terminal test | True for checkmate, stalemate and any applicable draw condition |
| Utility | Large positive value for a win by MAX, large negative for a loss, zero for a draw |

## 2.5 Assumptions

- The opponent is assumed to play the move that is best for it according to the same evaluation (optimal-reply assumption).
- The rules of chess, including all special moves, are provided by a trusted rules component.

## 2.6 Constraints

- Computation is bounded by a depth limit and an optional time limit; a legal move is always returned.

## 2.7 Success Criteria

| Criterion | How it is measured | Target |
|---|---|---|
| Legality | Returned move checked against the legal-move list | 100% legal |
| Search correctness | Alpha-Beta root value compared with plain Minimax | Identical values |
| Efficiency | Nodes visited with and without pruning | Fewer or equal nodes |

# 3. Background and Adversarial Search

Chess is deterministic, zero-sum and has perfect information. Minimax models opposition with MAX nodes that select the highest child score and MIN nodes that select the lowest.

## 3.1 Concepts

| Concept | Meaning | Role |
|---|---|---|
| State | Position plus metadata | Tree node |
| Action | Legal move | Tree edge |
| MAX | AI to move | Maximum child |
| MIN | Opponent to move | Minimum child |
| Terminal | Mate/draw | Rule score |
| Heuristic leaf | Depth frontier | Position estimate |

## 3.2 Search versus evaluation

Minimax and Alpha-Beta govern exploration; the evaluator estimates leaf quality. A strong search cannot compensate for illegal moves or incorrect terminal handling.

## 3.3 Design choice

Use depth-limited Minimax with Alpha-Beta as the core; add move ordering, iterative deepening and transposition caching as enhancements.

## 3.4 Game Tree and Branching Factor

The game tree has the current position as its root. Every legal move leads to a child node, and the children of a node alternate between the two players. Two numbers describe its size: the branching factor $b$, the average number of legal moves per position, and the depth $d$, the number of plies searched. A tree of depth $d$ has about $b^d$ leaves, which explains why even a small increase in depth is costly.

## 3.5 Why Minimax Gives a Good Move

Minimax assumes that after each of its own moves the opponent will choose the reply that is worst for MAX. The value of a MAX node is therefore the best of the values its children can guarantee, and the value of a MIN node is the worst of them. Under this assumption the root move chosen is optimal with respect to the search depth and evaluation used; if the opponent actually plays a weaker reply, the outcome for MAX can only be at least as good as the backed-up value predicts.

## 3.6 Related Search Methods

| Method | Idea | Relation to this project |
|---|---|---|
| Negamax | Single recursive function using the identity max(a, b) = -min(-a, -b) | Equivalent to Minimax with simpler code |
| Iterative deepening | Repeat the search with depth 1, 2, 3, ... and keep the last complete result | Planned enhancement; supports time limits |
| Quiescence search | Extend the search at unstable leaves until captures settle | Reduces the horizon effect |

## 3.7 Connection to Other Course Topics

- **State-space search:** the same ideas of states, actions and transition models apply, with an added opponent.
- **Heuristics:** the evaluator plays the role of a heuristic estimate, but it scores desirability rather than distance to a goal.
- **Algorithm analysis:** comparing $O(b^d)$ with $O(b^{d/2})$ shows how pruning changes growth, not just constants.

# 4. Requirements and Scope

## 4.1 Functional requirements

- Parse a valid position and generate only legal moves.
- Alternate MAX and MIN turns.
- Evaluate non-terminal frontier positions.
- Apply Alpha-Beta bounds and cutoffs.
- Detect checkmate, stalemate and draws.
- Return best move and search statistics.
- Support depth/time limits and handle no-legal-move positions.

## 4.2 Non-functional requirements

| Requirement | Design target |
|---|---|
| Correctness | Tested rules and terminal states |
| Efficiency | Pruning and move ordering |
| Responsiveness | Depth/time limits |
| Maintainability | Separate rules, search, evaluator, UI |
| Explainability | Move, score, depth, counters |

## 4.3 Scope

This is a proposed prototype, not a claim of grandmaster strength. Use a tested chess rules library unless implementing move generation is itself required.

## 4.4 Assumptions and Constraints

- The prototype plays one side at a time and analyses a single position per request.
- Search depth is fixed by configuration, with an optional time limit; no randomness is used, so tests are repeatable.

## 4.5 Suggested Tools and Environment

| Component | Suggested choice | Reason |
|---|---|---|
| Language | Python (prototype); Java or C++ for speed | Python is quick to write and test; compiled languages search deeper |
| Rules engine | python-chess library [5] | Tested move generation, check and draw detection |
| Test framework | pytest or unittest | Automates the functional tests listed in Section 13 |
| Notation | FEN for positions, SAN or UCI for moves | Standard, human-readable and library supported |

## 4.6 Acceptance Criteria

- Every returned move appears in the legal-move list for the supplied position.
- Checkmate, stalemate and the configured draw rules are recognised in all test positions.
- Alpha-Beta and plain Minimax return identical root values on every comparison position.

## 4.7 Requirement Priorities

| Priority | Requirements | Rationale |
|---|---|---|
| Must have | Legal moves, MAX/MIN alternation, evaluation, Alpha-Beta, terminal detection | Without these no valid decision can be produced |
| Should have | Move ordering, depth and time limits, statistics report | Make the search efficient and explainable |

# 5. System Architecture

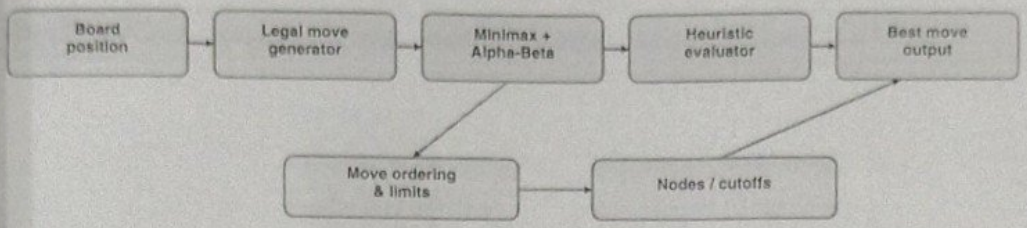



Figure 1. High-level chess decision-making pipeline.

## 5.1 Modules

| Module | Responsibility |
|---|---|
| Board interface | Load and display position |
| Rules engine | Generate legal moves and apply/undo |
| Terminal detector | Mate, stalemate and draw checks |
| Move ordering | Prioritise promising moves |
| Minimax | Propagate MAX/MIN scores |
| Alpha-Beta | Track bounds and prune |
| Evaluator | Score material and positional features |
| Reporter | Return move and statistics |

## 5.2 Data flow

The board enters legal-move generation. Each move creates a child position. Terminal states receive rule-based scores; frontier states receive heuristic scores. Values propagate upward, cutoffs reduce exploration, and the root move with the best value is returned.

## 5.3 Design Principles

- **Separation of concerns:** rules, search and evaluation are independent modules, so each can be tested alone.
- **Replaceable parts:** the evaluator can be swapped for a stronger one without modifying the search code.
- **Observability:** counters for nodes, leaves, terminals and cutoffs are updated in one place and reported at the end.

## 5.4 Module Interfaces

| Interface | Input | Output |
|---|---|---|
| Rules engine | Position | List of legal moves; a new position after make/undo |
| Terminal detector | Position | Not terminal, win, loss or draw |
| Move ordering | Position and legal moves | The same moves in a promising order |
| Evaluator | Non-terminal position | Integer score from MAX's perspective |
| Search | Position, depth, limits | Best move, value and statistics |

## 5.5 Extensibility

Because every module communicates through the small interfaces above, enhancements such as a transposition table, quiescence search or a learned evaluation function can be added by replacing or wrapping a single module. The search routine only needs to know how to obtain legal moves, how to apply and undo them, and how to score a position; it is unaware of how those services are implemented.

# 6. Board State and Legal Move Generation




Figure 2. Schematic board state and legal-move generation.

## 6.1 Board state

A position requires the 64 squares plus side to move, castling rights, en-passant target and relevant counters. FEN is a common compact representation.

## 6.2 Legal moves

- A legal move follows piece rules and leaves the moving king safe.
- Castling requires valid rights and safe king transit squares.
- En passant and promotion must be handled correctly.
- Moves exposing the king to attack are rejected.
- No legal moves in check means checkmate; without check means stalemate.

## 6.3 Apply and undo

Undo must restore captures, castling rights, en-passant data, clocks and side to move.

## 6.4 Example FEN String

The standard starting position is written in FEN as shown below. FEN lists the ranks from eight down to one, uses lower-case letters for black pieces, upper-case for white pieces and digits for runs of empty squares.

```
rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
```

| Field | Example | Meaning |
|---|---|---|
| Piece placement | rnbqkbnr/.../RNBQKBNR | Contents of ranks 8 to 1 |
| Side to move | w | White to move (b for Black) |
| Castling rights | KQkq | Both sides may still castle on either wing |
| En-passant target | - | No en-passant capture is currently available |
| Half-move clock | 0 | Moves since the last capture or pawn move (fifty-move rule) |
| Full-move number | 1 | Incremented after Black moves |

## 6.5 Validating Move Generation

Move generators are easy to get subtly wrong, especially for castling, en passant and promotion. A standard check is a *perft* test, which counts all leaf positions reachable in a given number of plies and compares the count with published reference values. From the initial position the expected counts are:

| Depth (plies) | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Leaf positions | 20 | 400 | 8,902 | 197,281 | 4,865,609 |

Any mismatch indicates a rules error that must be corrected before search results can be trusted.

# 7. Heuristic Evaluation Function

A heuristic estimates non-terminal positions at the depth limit. Positive scores favour MAX; negative scores favour the opponent. Centipawns are common, with 100 points approximately equal to one pawn.

## 7.1 Example formula

Eval(s) = Material + PieceActivity + CentreControl + PawnStructure + KingSafety

| Component | Illustrative method | Purpose |
|---|---|---|
| Material | Pawn 100, knight 320, bishop 330, rook 500, queen 900 | Material balance |
| Activity | Mobility bonuses | Piece placement |
| Centre | Central control | Influence |
| Pawn structure | Doubled/isolated penalties | Weaknesses |
| King safety | Exposure and defenders | Protection |
| Terminal | Mate/draw score | Rule outcome |

## 7.2 Consistency

Evaluate from MAX's fixed perspective when alternating MAX/MIN. Negamax is an alternative, but score perspective must remain consistent.

## 7.3 Terminal scores

Checkmate scores must dominate ordinary evaluation. Draws use the engine's draw policy.

## 7.4 Evaluation Features in More Detail

- **Material:** sum of piece values for MAX minus the same for MIN. It is usually the largest term, which is why a lost queen is penalised so heavily.
- **Piece activity:** reward pieces that control many squares or stand on strong squares; piece-square tables assign a small bonus or penalty to each piece type on each square.
- **King safety:** pawn shelter and nearby defenders matter in the middlegame, while an active king becomes an asset in the endgame.

## 7.5 Worked Evaluation (Hypothetical Values)

nvented values for a position in which MAX is a pawn ahead; not computed from a real board.

| Component | Score (centipawns) | Comment |
|---|---|---|
| Material | +100 | One extra pawn |
| Piece activity | +12 | Slightly more mobile pieces |
| Centre control | +8 | Two pieces influence the centre |
| Pawn structure | -15 | One doubled pawn |
| King safety | -10 | Exposed king-side shelter |
| **Total** | **+95** | Favours MAX by about one pawn |

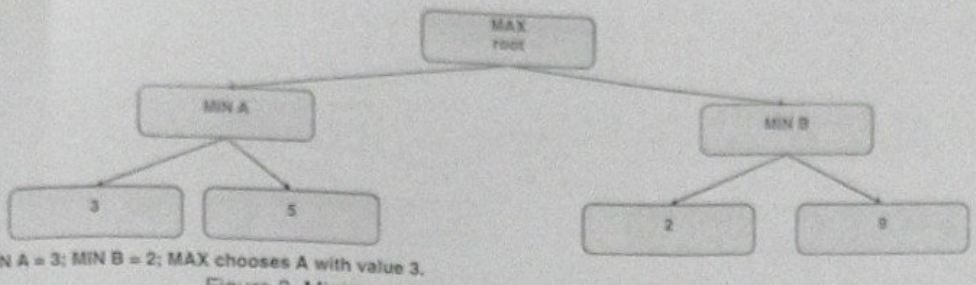
## .6 Tuning and Game Phase

Veights are normally tuned by testing: change one weight, analyse many positions and keep the change if esults improve. Many engines also blend middlegame and endgame scores by remaining material.

## .7 Terminal Score Convention

common approach assigns checkmate a value far outside the normal evaluation range, for example 100000 inus the ply at which mate occurs. Subtracting the ply makes the search prefer a faster mate and delay an navoidable loss. Stalemate and other draws receive the draw score, typically zero.

# 8. Minimax and Alpha-Beta Search




MIN A = 3; MIN B = 2; MAX chooses A with value 3.

Figure 3. Minimax score propagation on a small illustrative tree.

## 8.1 Minimax rule

At MAX, value = max(child values); at MIN, value = min(child values). In the diagram A=min(3,5)=3, B=min(2,9)=2, so MAX selects A with value 3.

## 8.2 Alpha-Beta bounds

Alpha is the best value MAX can guarantee; beta is the best value MIN can guarantee. Update alpha at MAX and beta at MIN. When alpha ≥ beta, remaining children cannot change the ancestor decision and may be pruned.

## 8.3 Complexity

Minimax worst case is O(b^d). With ideal ordering, Alpha-Beta can approach O(b^(d/2)); actual gains depend on move ordering.

## 8.4 Properties of Minimax

| Property | Value | Explanation |
|---|---|---|
| Completeness | Yes (finite tree) | A finite game tree is searched fully to the depth limit |
| Optimality | Against an optimal opponent | Backed-up values assume the opponent replies optimally |
| Time | $O(b^d)$ | Every node to depth d is examined in the worst case |
| Space | O(b x d) | Depth-first search stores only the current path and siblings |

## 8.5 Why Pruning Is Safe

Suppose MAX has already found a move worth 3. While examining another move, MIN finds a reply worth 2. MIN would choose that reply or something even lower, so this second move cannot be worth more than 2 to MAX. The remaining replies cannot raise the value above 2, so examining them would never change the decision. The same argument applies symmetrically at MIN nodes. Pruning therefore saves work without altering the root value.

## 8.6 Effect of Move Ordering

The number of cutoffs depends strongly on the order in which moves are tried. If the best move is examined first at every node, Alpha-Beta needs roughly $b^{d/2}$ leaves instead of $b^d$. If the worst move is examined first, no pruning occurs and the search degrades to plain Minimax. For b = 35 and d = 6, plain Minimax examines about 1.8 billion leaves, while the best case examines on the order of $10^5$, as analysed by Knuth and Moore [3].

| Ordering quality | Cutoffs | Approximate cost |
|---|---|---|
| Best move first (ideal) | Maximum | About $b^{d/2}$ |
| Good heuristic ordering (captures first) | Many | Close to the ideal in practice |
| Random ordering | Some | Between ideal and worst case |
| Worst move first | None | About $b^d$ (same as Minimax) |

## ). Proposed Algorithm and Pseudocode

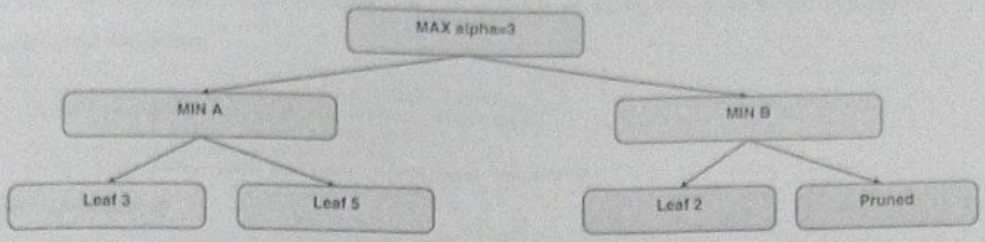



Figure 4. Alpha-Beta cutoff example.

### 1 Steps

Check terminal state; return mate/draw score.

Evaluate at depth limit.

Generate and order legal moves.

Update alpha at MAX and beta at MIN.

Stop siblings when alpha ≥ beta.

Undo moves, propagate scores and keep the best root move.

### 2 Pseudocode

```
FUNCTION alphabeta(pos, depth, alpha, beta, maxNode):
  if terminal(pos): return terminal_score(pos)
  if depth == 0: return evaluate(pos)
  moves = order(legal_moves(pos))
  if maxNode:
    value = -INF
    for move in moves:
      make(move); child = alphabeta(pos, depth-1, alpha, beta, false); undo(move)
      value = max(value, child); alpha = max(alpha, value)
      if alpha >= beta: break
    return value
  else:
    value = +INF
    for move in moves:
      make(move); child = alphabeta(pos, depth-1, alpha, beta, true); undo(move)
      value = min(value, child); beta = min(beta, value)
      if alpha >= beta: break
    return value
```

### 3 Root Search with Iterative Deepening

```
FUNCTION search_root(pos, maxDepth, timeLimit):
  bestMove = null
  for depth = 1 to maxDepth:
    if time_expired(timeLimit): break
    alpha = -INF; beta = +INF; roundBest = null
    for move in order(legal_moves(pos)):
      make(move)
      value = alphabeta(pos, depth-1, alpha, beta, false)
      undo(move)
      if value > alpha: alpha = value; roundBest = move
    if round_completed(): bestMove = roundBest
  return bestMove, alpha, statistics
```

### 4 Implementation Notes

Make and undo must be exact inverses; a single undo bug corrupts every later result.

Count nodes, leaves, terminals and cutoffs inside the recursive function so the statistics are always consistent.

Discard the result of an interrupted iteration; only completed iterations may change the chosen move.

# 10. Implementation Design

A prototype can use Java, Python or C++. Separate rules from search so the algorithms can be tested independently.

## 10.1 Suggested modules

```
chess-ai/
  board/ position, legal_moves, make_unmake, terminal
  engine/ evaluate, move_ordering, minimax, alpha_beta
          transposition_table, search_stats
  interface/ board_view, game_controller
  tests/ legal_moves, mate_stalemate, alpha_beta, evaluation
```

## 10.2 Search statistics

| Statistic | Definition |
|---|---|
| Nodes visited | Recursive nodes entered |
| Leaves evaluated | Frontier positions scored |
| Terminal nodes | Mate/draw resolved by rules |
| Cutoffs | Sibling exploration stopped |
| Depth completed | Deepest complete iteration |
| Elapsed time | Wall-clock duration |

## 10.3 Improvements

- Move ordering: examine captures and promotions early.
- Iterative deepening: keep the last completed result.
- Transposition table: reuse repeated positions.
- Quiescence search: extend unstable tactical leaves.
- Time control: return a valid move on timeout.

## 10.4 Illustrative Python Sketch

The fragment below shows how the search could be written with the python-chess library [5]. Scores are kept from White's perspective, so White is MAX. It is a sketch for explanation, not a tuned engine.

```
import chess
MATE, INF = 100000, 10**9
VALUE = {chess.PAWN: 100, chess.KNIGHT: 320, chess.BISHOP: 330,
         chess.ROOK: 500, chess.QUEEN: 900, chess.KING: 0}

def evaluate(board):
    score = 0
    for sq, piece in board.piece_map().items():
        v = VALUE[piece.piece_type]
        score += v if piece.color == chess.WHITE else -v
    return score

def alphabeta(board, depth, alpha, beta, maxing, ply=0):
    if board.is_checkmate():
        return (-MATE + ply) if maxing else (MATE - ply)
    if board.is_stalemate() or board.is_insufficient_material():
        return 0
    if depth == 0:
        return evaluate(board)
    best = -INF if maxing else INF
    for move in sorted(board.legal_moves, key=board.is_capture, reverse=True):
        board.push(move)
        val = alphabeta(board, depth - 1, alpha, beta, not maxing, ply + 1)
        board.pop()
        if maxing: best = max(best, val); alpha = max(alpha, best)
        else:      best = min(best, val); beta = min(beta, best)
        if alpha >= beta: break
    return best
```

# 11. Worked Example and Sample Input

This fictional tree demonstrates mechanics, not a full chess position. Leaf values are hypothetical evaluations from MAX's perspective.

## 11.1 Sample input

```
Current position P0: A1 is MAX
Depth: 2 plies after root
Root moves: A, B
Opponent replies: A -> A1,A2; B -> B1,B2
Leaf scores: A1=3, A2=5, B1=2, B2=9
```

## 11.2 Minimax calculation

| Node | Children | Operation | Value |
|---|---|---|---|
| A (MIN) | 3, 5 | min(3,5) | 3 |
| B (MIN) | 2, 9 | min(2,9) | 2 |
| Root (MAX) | A=3, B=2 | max(3,2) | 3 |

## 11.3 Alpha-Beta walkthrough

Search A first: leaves 3 and 5 give A=3, so root alpha becomes 3. Search B: its first leaf is 2, so beta becomes 2. Since beta ≤ alpha, MAX already has a better option and B's remaining leaf 9 can be pruned.

The cutoff depends on searching leaf 2 before leaf 9. Different ordering can change pruning count, not the final Minimax value.

## 11.4 Effect of Searching in a Different Order

The final Minimax value is independent of move order, but the amount of pruning is not. The table repeats the same tree with two root orderings, using the leaf scores A1 = 3, A2 = 5, B1 = 2 and B2 = 9.

| Root order | Leaves evaluated | Leaves pruned | Root value |
|---|---|---|---|
| A first, then B | A1, A2, B1 (3 leaves) | B2 | 3 |
| B first, then A | B1, B2, A1, A2 (4 leaves) | None | 3 |

When B is searched first, its value is 2 and the root alpha becomes 2. Searching A afterwards, the first leaf gives A a bound of 3, which is still above alpha, so no cutoff is possible and every leaf must be examined.

## 11.5 Alpha and Beta Trace for Order A, B

| Step | Node | Alpha | Beta | Action |
|---|---|---|---|---|
| 1 | Root (MAX) | -INF | +INF | Begin with the full window |
| 2 | A (MIN), leaf A1 = 3 | -INF | 3 | Beta tightens to 3 |
| 3 | A (MIN), leaf A2 = 5 | -INF | 3 | min(3, 5) = 3, no change |
| 4 | Root after A | 3 | +INF | Alpha becomes 3 |
| 5 | B (MIN), leaf B1 = 2 | 3 | 2 | Alpha is at least beta: cutoff |
| 6 | Root after B | 3 | +INF | B is at most 2, so A remains best |

## 11.6 Chess Interpretation

In a real position, A and B would be two candidate moves for MAX, and each reply would lead to a new position scored by the evaluator. Move B could, for example, look attractive at first but allow the opponent a reply that wins material. Once that single refutation is found, there is no need to look for further replies, because MAX already has a better alternative in move A.

## 11.7 Key Observations

- A MIN node can be cut off as soon as one reply is no better for MAX than an option MAX already has.
- Pruned leaves are not guessed: their values are provably irrelevant to the final decision.

# 12. Sample Output and Interpretation

## 12.1 Search trace

| Step | Action | Value/bound | Result |
|---|---|---|---|
| 1 | Evaluate A1 | 3 | MIN A best=3 |
| 2 | Evaluate A2 | 5 | A=3 |
| 3 | Return A | 3 | Root alpha=3 |
| 4 | Evaluate B1 | 2 | MIN B beta=2 |
| 5 | Compare | alpha=3, beta=2 | Cutoff |
| 6 | Skip B2 | not evaluated | One leaf pruned |
| 7 | Root | A=3, B≤2 | Choose A |

## 12.2 Example output

```
Search completed: yes
Depth: 2 plies after root
Root moves: 2
Leaf evaluations: 3
Pruned leaves: 1
Move A value: 3; Move B bound: <= 2
Selected best move: Move A
```

## 12.3 Interpretation

Counts depend on terminal checks, caching, move ordering and counter definitions. A pruned branch was not searched because the bound proves it cannot change the decision.

## 12.4 Chess notation

Return SAN (e.g. Nf3) or coordinate notation (e.g. g1f3), matched to the legal move list.

## 12.5 Comparison of Plain Minimax and Alpha-Beta on the Example

| Statistic | Plain Minimax | Alpha-Beta (order A, B) |
|---|---|---|
| Root value | 3 | 3 |
| Leaf evaluations | 4 | 3 |
| Nodes visited (root, A, B and leaves reached) | 7 | 6 |
| Cutoffs | 0 | 1 |
| Leaves saved | 0 | 1 (25 percent) |

Both methods select Move A. On a tree this small the saving is modest, but the proportion of work avoided grows quickly as depth and branching increase.

## 12.6 Format of a Larger Report (Illustrative Values Only)

For a full chess position, the reporter would print the same fields with larger counters. The values below are invented to show the format and are not measurements.

```
Position: <FEN supplied by user>
Search completed: yes          Depth completed: 4 plies
Nodes visited: 18,452          Leaves evaluated: 12,907
Terminal nodes: 6              Cutoffs: 2,311
Elapsed time: 0.42 s           Nodes per second: 43,933
Best move (SAN): Nf3           Value: +35 centipawns
```

## 12.7 How to Read the Output

- **Value** is from MAX's perspective: positive favours the AI, negative favours the opponent.
- **Depth completed** tells the reader how far ahead the decision looked, which matters when a time limit applies.
- **Cutoffs** and **nodes visited** together show how effective the pruning and move ordering were.

# 13. Testing and Evaluation Plan

Test rule correctness and search correctness separately. Hand-checkable trees verify score propagation; known positions test legal moves and terminal conditions.

### 13.1 Functional tests

| Test | Scenario | Expected |
|---|---|---|
| T1 | Initial board | Only legal opening moves |
| T2 | Move exposes king | Reject |
| T3 | Checkmate | Mate score |
| T4 | Stalemate | Draw score |
| T5 | Castling/en passant | Rules correct |
| T6 | Promotion | Valid choices |
| T7 | Known tree | Manual Minimax match |
| T8 | Alpha-Beta | Same root value, fewer/equal nodes |
| T9 | Move ordering change | Same value; cutoffs may differ |
| T10 | Timeout | Last completed result |

### 13.2 Metrics

- Nodes per second, total nodes and leaves evaluated.
- Alpha-Beta cutoffs, elapsed time and completed depth.
- Move legality and terminal-state correctness.
- Playing strength in controlled matches.

### 13.3 Baselines

Compare plain Minimax and Alpha-Beta on identical positions/depths. Test strength against a simple material-only player or recognised engine under controlled settings.

### 13.4 Test Procedure

- **Step 1 – Rules first:** run legal-move and perft tests; do not proceed until all counts match.
- **Step 2 – Search on hand-made trees:** check Minimax and Alpha-Beta against the worked example and other small trees.
- **Step 3 – Equivalence:** on many positions, confirm that both algorithms return the same root value.
- **Step 4 – Performance:** record nodes, cutoffs and time with and without move ordering.

### 13.5 Expected Trends

| Change | Expected effect | Reason |
|---|---|---|
| Increase depth by one ply | Nodes grow by roughly the branching factor | Tree grows geometrically |
| Enable Alpha-Beta | Fewer nodes, same root value | Irrelevant branches are skipped |
| Improve move ordering | More cutoffs, deeper search in same time | Strong bounds appear earlier |
| Enable iterative deepening | Slight overhead, better ordering and time control | Earlier rounds guide later ones |

### 13.6 Recording Results

Each experiment should record the position, depth, settings, node counts, elapsed time and chosen move so that results can be reproduced. Tables of nodes against depth, with and without pruning, make the benefit of Alpha-Beta visible, and any disagreement between root values is treated as a defect to be fixed.

# 14. Limitations and Responsible Design

Chess has a large branching factor. Shallow search may miss tactics, and a simple evaluator can misjudge positional compensation. Alpha-Beta improves efficiency but does not improve evaluation quality by itself.

## 14.1 Limitations and mitigations

| Limitation | Mitigation |
|---|---|
| Large branching factor | Alpha-Beta, ordering, iterative deepening |
| Horizon effect | Quiescence search |
| Weak evaluator | Tune and validate features |
| Move-generator bugs | Trusted library and perft tests |
| Repeated positions | Repetition checks and transposition table |
| Draw errors | Implement applicable draw rules |
| Timeout | Return last completed result |

## 14.2 Terminal and draw handling

Distinguish checkmate from stalemate and recognise applicable draws. Repetition may require move history; a board snapshot alone may not be enough.

## 14.3 Future enhancements

Opening books, endgame tablebases, richer evaluation, quiescence search, transposition tables, parallel search and neural evaluation can be added after the core passes tests.

## 14.4 The Horizon Effect

A fixed-depth search treats positions at the depth limit as if the game paused there. If a capture or threat is just beyond that limit, the evaluator may score an unstable position as safe, or the search may delay a loss by making pointless forcing moves so that it falls beyond the horizon. Quiescence search reduces this by continuing to search captures and similar forcing moves until the position is quiet before the evaluator is called.

## 14.5 Evaluator Bias

Because the search maximises the evaluator's score, any systematic error in the evaluator is exploited by the search. For example, an evaluation that overvalues piece mobility may lead the program to sacrifice material for activity it cannot convert. Evaluation features should therefore be validated against test positions rather than assumed correct.

## 14.6 Responsible Use and Transparency

- **Fair play:** the program is intended for learning, analysis and permitted engine play; it must not be used to cheat in human competitions or on platforms that prohibit engine assistance.
- **Honest reporting:** results should state the depth, time limit and baseline used, and should not overstate strength beyond what the controlled tests show.
- **Attribution:** ideas, libraries and reference values taken from other sources should be cited, as in the references of this report.

## 14.7 Summary of Risks

| Risk | Effect if ignored | Control |
|---|---|---|
| Rules bug | Illegal or wrong moves | Trusted library, perft tests |
| Evaluator error | Systematically poor decisions | Validate and tune features |
| Insufficient depth | Missed tactics | Ordering, iterative deepening, quiescence |

# 5. Conclusion and References

## 5.1 Conclusion

[illegible] proposed chess game-playing using Minimax and Alpha-Beta search. It models positions as states, generates legal [illegible] positions, alternates MAX/MIN choices and evaluates frontier positions heuristically. The example selected Move A with [illegible] 3 and showed how Alpha-Beta can prune a branch once its value is bounded at 2 or less.

[illegible] steps are to implement a tested rules layer, evaluator and search routine; verify against hand-calculated trees; compare [illegible] counts with and without pruning; and add move ordering and iterative deepening. Correct rule handling and terminal [illegible] matter as much as search speed.

## 5.2 References


[illegible] Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021.

[illegible] E. Shannon, "Programming a Computer for Playing Chess," Philosophical Magazine, vol. 41, no. 314, 1950.

[illegible] E. Knuth and R. W. Moore, "An Analysis of Alpha-Beta Pruning," Artificial Intelligence, vol. 6, no. 4, 1975.

Chess Programming Wiki: Minimax, Alpha-Beta, Evaluation and Move Ordering. https://www.chessprogramming.org/

python-chess documentation. https://python-chess.readthedocs.io/

FIDE. Laws of Chess. https://handbook.fide.com/


[illegible] leaf values are illustrative and not scores from a real chess position.

## 5.3 Learning Outcomes Achieved

Formulated chess as an adversarial search problem with states, actions, terminal tests and utilities.

Analysed Minimax and Alpha-Beta pruning and traced both by hand on a small tree.

Related search efficiency to branching factor, depth and move ordering.

Designed a modular architecture, test plan and evaluation approach for a prototype.

## 5.4 Suggested Future Work

| Stage | Task | Outcome |
|---|---|---|
| 1 | Implement the rules layer and perft tests | Verified legal-move generation |
| 2 | Implement Minimax, Alpha-Beta and the evaluator | Working fixed-depth player |
| 3 | Add move ordering and iterative deepening | Greater depth within the time limit |
| 4 | Add transposition table and quiescence search | Fewer repeated nodes, fewer tactical errors |

## Appendix A: Glossary

| Term | Definition |
|---|---|
| Alpha | Best value MAX is currently guaranteed along the path to the root |
| Beta | Best value MIN is currently guaranteed along the path to the root |
| Branching factor | Average number of legal moves available in a position |
| Centipawn | One hundredth of a pawn, the usual unit for evaluation scores |
| Horizon effect | Errors caused by stopping the search just before a critical event |
| Perft | Count of reachable positions at a depth, used to validate move generation |
| Ply | A single move by one player |
| Transposition | Reaching the same position by different move orders |